

INTERVIEW PREP NOTES

Backend Interview Prep

Every claim on my resume, explained from zero. Written in the hours before an interview — the questions that actually get asked, and the ones that expose whether you really know it.

What's inside

The danger ranking — which resume claims fall apart under a follow-up question • the top 30 questions, plus 20 project, 20 Java/Spring, 15 React, 10 SQL, 10 security and 10 ML questions • the 10 questions that expose bluffing, answered honestly • the ten "why?" and ten "what if?" questions • three 30-second project scripts • a 15-minute final revision plan

Written for anyone preparing for a first Java or backend interview. If it helps you, use it.

Siva Charan Kollauram Gandla • Software Developer • Final-year Computer Science

WHAT THIS IS

I had an interview the next morning and three hours to prepare. So I did the thing that actually works: I took my own resume and forced myself to explain **every single claim** from zero — no textbook definitions, only the parts that get tested in the room.

- **Why** did you use it?
- **What** happens internally?
- **What if** it breaks?
- **Show** me the code.

Written for a fresher Java/backend interview. If you're preparing for your first one, this is the version I wish someone had handed me.

The most useful section is the last one — the questions that expose bluffing, answered honestly instead of hidden.

THE FOUR IDEAS THIS WHOLE DOCUMENT RESTS ON

1. The architecture spine. Almost every backend question collapses into this one diagram:

```
REACT —JSON + JWT→ CONTROLLER → SERVICE → REPOSITORY → JPA/HIBERNATE → POSTGRES  
(UI)           (the door)   (the rules)   (the SQL)   (maps objects)   (tables,
```

	+ the	made for	to rows)	rows, keys)
	transaction)	you)		

What each box would say if it had one sentence:

Layer	One sentence
Controller	"I'm the door. I read the URL, headers and JSON, validate it, and I contain zero business rules."
Service	"I'm the brain and I draw the transaction boundary. Either all my writes commit, or none do."
Repository	"I'm an interface. Spring writes my implementation — I declare <i>what</i> I need, not <i>how</i> ."
JPA / Hibernate	"I map Java objects to table rows and generate the SQL nobody wants to type."
PostgreSQL	"I'm the last line of defence. Even a buggy service can't insert a duplicate — the constraint stops it."

2. The security spine.

"Login returns a **signed** JWT containing the user id, the role and an expiry. On every later request the client sends `Authorization: Bearer <token>`. A filter validates the signature and expiry and puts the authenticated user into the security context. Then path rules check roles, and the service additionally checks **ownership** — because a BUYER role only means *this* buyer can act on *their own* transaction. Bad or expired token → 401. Valid user, wrong transaction → 403."

3. JWT is signed, NOT encrypted. Say this correctly and you pass a question most candidates fail.

4. Never bluff. "I don't know, but here's how I'd find out" scores 3–4 out of 5. Bluffing scores 0 and taints everything after it.

THE DANGER RANKING

Claims where the question "how?" breaks you. Ranked by how fast they expose depth.

DANGER: EXTREME

- 1. JWT authentication + role-based authorization** Easy to fake, easy to destroy. **You must know:** header/payload/signature · signed not encrypted · the server verifies signature + expiry · `Authorization: Bearer` header · 401 vs 403 · roles plus ownership checks. **Never say:** "JWT is encrypted."
- 2. Escrow workflow with six states** If "escrow" is fuzzy, the flagship project collapses. **You must know:** escrow = a neutral party holds funds until conditions are met · a state machine enforces only legal transitions · the server decides the next state, the client never posts a status.
- 3. "Designed entity relationships"** They will ask you to draw it live. **You must know:** users relate to transactions twice (buyer, seller) · one transaction has one escrow and many history rows · ledger entries reference the transaction · `@ManyToMany` owns the foreign key.
- 4. Spring Data JPA + JPA/Hibernate** ORM is where beginners get destroyed. **You must know:** JPA is the specification, Hibernate is the implementation, Spring Data JPA generates the repositories · `@Entity` → table · the persistence context is a per-transaction cache.

DANGER: HIGH

5. Random Forest — "Why not one tree?" Two sources of randomness: bootstrap sampling of rows, and a random feature subset at each split. Averaging decorrelated trees lowers variance.

6. Transactions on money movement — "What if two requests arrive together?" The transaction's state check rejects one; an `@Version` optimistic-lock column makes the loser fail with a 409. Never a silent double release.

7. "500+ LeetCode problems" — They can hand you a problem right now. Be able to do two-sum with a hash map, out loud, with the complexity.

8. React + hooks — Fundamental questions are binary. You either know props vs state or you don't.

DANGER: MEDIUM

9. Flask + scikit-learn integration — "Why two runtimes?" The ML ecosystem is Python-first; the web layer wants Java's typing, transactions and security. One entry point: the browser calls Java, Java calls Python.

10. Vite and Tailwind — Easy wins. Answer well in 30 seconds and don't oversell.

TOP 30 QUESTIONS I MUST KNOW

1. **Tell me about yourself** — 60 seconds: name, degree, CGPA, three projects, stack, what you want.
2. **Explain your strongest project** — the one-minute version.
3. **What is OOP?** — Data and behaviour bundled in objects, with encapsulation, abstraction, inheritance and polymorphism to keep change contained.
4. **Four pillars?** — Encapsulation: hide internals, control legal changes. Abstraction: a simple contract over complexity. Inheritance: reuse via is-a. Polymorphism: one call, correct behaviour per type.
5. **What is Java?** — A language plus a runtime: compile to bytecode once, the JVM runs it anywhere.
6. **JDK / JRE / JVM?** — Dev tools / runtime environment / the engine that runs bytecode.
7. **== vs equals()?** — `==` is identity (reference); `equals()` is content. Use `equals()` for strings.
8. **Why is String immutable?** — Security, string pooling, safe HashMap keys, thread safety.
9. **Collections?** — List (ordered, duplicates), Set (unique), Map (key to value). ArrayList `get` is $O(1)$; HashMap lookup is $O(1)$ average.
10. **Interface vs abstract class?** — A capability contract with multiple inheritance vs shared state and code with a single parent.
11. **What is Spring?** — A Java framework whose container creates and injects your objects instead of you wiring them.
12. **What is Spring Boot?** — Spring without manual configuration: auto-configuration, starters, an embedded Tomcat.
13. **Dependency injection?** — Dependencies provided from outside, usually via the constructor — explicit, final, and testable without Spring.
14. **What is a bean?** — An object the Spring container manages. Singleton by default.
15. **What is a REST API?** — Resources with URLs, standard HTTP methods, stateless requests, JSON representations.
16. **GET vs POST?** — GET reads and must be safe and idempotent. POST creates or triggers an action and is not idempotent.
17. **Status codes?** — 200/201/204 OK-created-no content · 400 bad input · 401 not authenticated · 403 not permitted · 404 missing · 409 conflict with current state · 500 server bug.
18. **JPA vs Hibernate vs Spring Data JPA?** — Specification, implementation, repository abstraction.
19. **What is ORM?** — Mapping classes to tables and objects to rows, generating SQL so you work in objects.

20. **Entity relationships?** — `@ManyToOne` holds the foreign key; `@OneToMany` uses `mappedBy`; many-to-many needs a join table.
21. **What is `@Transactional`?** — Wraps the method in one database transaction: all changes commit or all roll back. Unchecked exceptions roll back by default.
22. **What is authentication?** — Proving who you are. Login, and the JWT.
23. **What is authorization?** — What you may do. Roles plus ownership checks.
24. **How does JWT work?** — `header.payload.signature`. The server signs with a secret and verifies signature plus expiry on each request. The payload is readable, not encrypted.
25. **Why BCrypt?** — One-way, salted, deliberately slow. A leaked table isn't crackable, and identical passwords produce different hashes.
26. **What is React?** — A library for state-driven UI with components. You describe the UI for the current state and React updates the DOM.
27. **Props vs state?** — Props come from the parent and are read-only. State is owned by the component and changing it re-renders.
28. **What is Random Forest?** — Many decision trees, each trained on a bootstrap sample with a random feature subset at each split. Probabilities are averaged, reducing variance.
29. **What is Big-O?** — How time or memory grows with input size, ignoring constants. $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$.
30. **Why should we hire you?** — Solid Java and Spring fundamentals, three projects where I had to make real design decisions, and a habit of prioritising correctness where a wrong answer is silent.

TOP 20 PROJECT QUESTIONS

1. **Walk me through your flagship project in a minute.**
2. **What is escrow and why does it exist?** — A neutral party holds the funds until the conditions are met, so neither side has to trust the other.
3. **Walk me through the escrow lifecycle.** — `created` → `funded` → `shipped` → `delivered` → `released`, with `dispute` → `review` → `resolve/refund`, plus scheduled expiry.
4. **What is the state machine and why?** — Legal transitions in one place, so no code path can set a status directly and impossible states don't exist.
5. **Two simultaneous release requests?** — The transaction's state check rejects one; `@Version` optimistic locking makes the loser fail with a 409. Never a silent double release.
6. **Why a ledger instead of a balance column?** — A stored balance can drift from the history it represents. Deriving it makes the ledger the single source of truth and auditing just re-running arithmetic.
7. **Draw your database schema.** — `users` → `transactions` (buyer and seller, twice) → `escrows`, `ledger_entries`, `transaction_history`.
8. **Which endpoints need which role?** — Parties act on their own transactions. Admin resolves reviews. Ownership is checked separately from role.
9. **What was hardest?** — Keeping state, ledger and audit consistent inside one transaction under concurrency.
10. **How did you handle money types?** — `BigDecimal` in Java, `NUMERIC(12,2)` in Postgres. Never floats.
11. **What happens if a token is invalid or expired?** — 401 from the entry point. The client clears the token and redirects to login.
12. **Walk me through an ML prediction end to end.** — `React` → `Spring` (auth, validate) → `Flask` (encode, predict, calibrate, top-5) → `Spring` stores history → `React`.
13. **Why Python for the model?** — The ML ecosystem is Python-first, and the fitted encoders stay next to the model that needs them.
14. **Why not call the model from the browser?** — One entry point means authentication and validation live in one place.
15. **How is your visualization project not just an animation?** — The frames are generated by the real algorithm implementations with stable value ids, and playback is data — seekable, steppable and

comparable.

16. **How does pause work?** — The reducer sets `playing: false`; the effect returns early and its cleanup clears the timer. Nothing is being paused — the algorithm already finished.

17. **What's the first thing you'd improve?** — Concurrency tests, an evaluation harness, compact event traces. Name a real improvement per project.

18. **Why separate frontend and backend?** — Two deployables, one contract. Each side changes and tests independently.

19. **What if the database constraint is violated?** — The database is the last line of defence. A unique constraint stops duplicates even when the service has a bug.

20. **What did you not have time to do?** — Answer honestly with your limitations list. Interviewers trust candidates who name limits.

TOP 20 JAVA / SPRING QUESTIONS

1. What is the JVM and why does bytecode exist?
2. `==` vs `equals()`, and the `hashCode` contract.
3. Why is String immutable?
4. ArrayList vs LinkedList — and when to use a Set instead of a List.
5. How does HashMap work, and what is the worst-case lookup?
6. Overloading vs overriding.
7. `final` on a variable, method and class.
8. Access modifiers — why fields are private.
9. Interface vs abstract class, and why Java has single inheritance but multiple interfaces.
10. Checked vs unchecked exceptions, and what `finally` guarantees.
11. What is `static`, and why can't a static method use an instance field?
12. What are generics for? — Compile-time type safety and no manual casting.
13. What is IoC and what is a bean?
14. Why constructor injection over field injection?
15. Why must a `@Service` be stateless? — Beans are singletons.
16. `@Component` vs `@Service` vs `@Repository` vs `@RestController`.
17. What does `@SpringBootApplication` combine, and is auto-configuration conditional?
18. Why can a `@Transactional` method silently lose its transaction? — Self-invocation bypasses the proxy.
19. Which exceptions roll back by default, and how do you change that?
20. What happens if two beans share a type? — `@Primary` or `@Qualifier`.

TOP 15 REACT QUESTIONS

1. What is React and what problem does it solve?
2. What is a component?
3. Props vs state.
4. What is JSX and what does it compile to?
5. Why can't you mutate state directly?
6. `useState` — and why the functional updater form?
7. `useEffect` — the three dependency-array cases.
8. Why does an effect need a cleanup function?
9. Why do keys matter, and what breaks with index keys?
10. When would you use `useReducer` instead of `useState`?

11. What is `useRef` for?
12. What is lifting state up?
13. Event handling and controlled inputs.
14. What does React Router give you?
15. What is the virtual DOM and reconciliation?

TOP 10 SQL QUESTIONS

1. WHERE vs HAVING — rows before grouping vs groups after aggregation.
2. INNER vs LEFT JOIN — and how a LEFT JOIN keeps users with zero rows.
3. Find duplicate emails.
4. Users with no transactions — an anti-join.
5. Primary key vs foreign key vs unique.
6. What is an index, and why do foreign keys need one in Postgres? (They aren't auto-indexed.)
7. What is a transaction, and what does ACID stand for?
8. Why `NUMERIC` and not `FLOAT` for money?
9. Write the ledger-balance check (`GROUP BY transaction_id HAVING SUM(...) <> 0`).
10. Normalisation — what 1NF/2NF/3NF prevent: duplication and update anomalies.

TOP 10 SECURITY QUESTIONS

1. Authentication vs authorization.
2. JWT structure and what each part does.
3. **Is a JWT encrypted?** — **No**. Signed and base64-encoded, readable by anyone. Its value is that it can't be *changed* undetected.
4. What exactly does the server verify per request? — Signature, expiry, and that the user still exists/is active.
5. Why BCrypt, and what does the salt do? — Random per password, stored inside the hash, defeats rainbow tables and makes identical passwords hash differently.
6. Why hashing instead of encryption for passwords? — Hashing is one-way, so a leak doesn't reveal passwords; encryption would need a key on the server that unlocks everything.
7. 401 vs 403 — which for a buyer touching someone else's transaction? — **403**. 401 means log in; 403 means you're recognised but this isn't yours.
8. Why is CSRF disabled, and is that safe? — A bearer token is attached explicitly by the client, not automatically by the browser, so the attack model doesn't apply to a stateless token API.
9. How do you log out with JWTs? — The client deletes the token. The honest limitation is that it stays valid until it expires, which is why expiry should be short.
10. Roles vs ownership — why a role check alone isn't authorization.

TOP 10 MACHINE LEARNING QUESTIONS

1. Traditional programming vs machine learning — you write the rules vs it learns the rules from labelled examples.
2. Feature, label, sample, model, prediction.
3. Classification vs regression — which is the job-role recommender? Multi-class classification.

4. What is preprocessing and why is it needed? — Models compute with numbers, not text, and messy data teaches garbage.
5. One-hot vs ordinal encoding — why not 1/2/3? Integer codes invent a false order and distance between categories.
6. Why fit transformers on the training set only? — **Data leakage**. Using test statistics makes the evaluation optimistic.
7. What is a decision tree and how does it choose a split? — By reducing impurity (Gini or entropy) the most.
8. What are the two sources of randomness in a Random Forest? — Bootstrap sampling of rows, and a random feature subset at each split.
9. How does a forest produce a probability? — Each tree outputs class probabilities; the forest averages them (soft voting).
10. Two advantages and two limitations — handles non-linearity and mixed types, gives feature importance / uncalibrated probabilities, less interpretable, weaker than boosting.

TOP 10 QUESTIONS THAT EXPOSE BLUFFING

Each with the safe, honest answer.

1. **"What accuracy did your model get?"** — No number is claimed. Describe the pipeline and say the evaluation harness (stratified validation, top-k accuracy, calibration metrics) is the next step.
2. **"Show me your test suite."** — Say exactly what exists and what is planned. Never say "fully tested."
3. **"Is your JWT encrypted?"** — No. Signed, readable payload, secret never in the client.
4. **"Where do you store the token, and is that safe?"** — Browser storage. The honest trade-off is XSS exposure versus the CSRF concerns of cookies.
5. **"Explain the persistence context and dirty checking."** — A per-transaction first-level cache that auto-flushes changes to managed entities. Stop there — don't invent eviction tuning.
6. **"How does your state machine handle concurrency?"** — `@Transactional` plus optimistic locking → the loser gets a 409. Don't claim pessimistic locking.
7. **"Is this deployed and used?"** — No. Repository-verified. Say so plainly.
8. **"Show me which ORM methods you wrote."** — Open the repository interface: derived query names plus a `@Query` with a join fetch.
9. **"Have you used Docker, Kubernetes, Kafka or Redis?"** — Not hands-on. A "no" costs nothing here because it isn't on your resume. Bluffing costs the interview.
10. **"You claim 500+ problems — solve this."** — Two-sum with a hash map, said out loud with the complexity, including the brute-force baseline.

Five things never to say:

1. "The accuracy was 94%."
2. "It's fully unit tested."
3. "JWT encrypts the payload."
4. "I deployed it and handled thousands of users."
5. "Yes" to a technology you can't then explain.

TOP 10 "WHY?" QUESTIONS

1. Why Spring Boot — auto-configuration, starters, an embedded server, so you write business logic instead of plumbing.
2. Why PostgreSQL — ACID transactions, real constraints, exact `NUMERIC` for money.
3. Why JWT and not sessions — the API is stateless and the identity has to travel with each request; the trade-off is revocation.

4. Why JPA/Hibernate and not raw JDBC — the mapping is declared once and you work with objects; the cost is hidden SQL, so you still need to understand SQL.
5. Why React — the UI is state-driven, which fits both role/status-dependent actions and frame-by-frame rendering.
6. Why Random Forest — small tabular data with mixed feature types; non-linearity and interactions for free; fast, with feature importances.
7. Why two services, Java and Python — each side uses the right ecosystem; you pay with a network hop and a contract to keep aligned.
8. Why a ledger instead of a balance column — a mutable balance can disagree with its history; derived balances can't.
9. Why a state machine — legality is enforced in one place instead of anywhere a status can be set.
10. Why record frames instead of animating execution — so playback is data, and seeking, pausing and stepping all become cursor moves.

TOP 10 "WHAT IF?" QUESTIONS

1. **Database is down** — a data-access exception, mapped to a 500 with a generic client message and details logged server-side; the transaction never starts, so nothing is half-written.
2. **The API call fails in React** — the shared API helper throws on non-2xx, the component renders an error state; 401 clears the token and returns to login.
3. **The JWT expires** — the server returns 401; the client treats it as session-expired. A short-lived access token plus a refresh token is the smoother design.
4. **The user is unauthorized** — 403, not 401. The difference matters to the client.
5. **Two release requests arrive together** — the state check under the transaction rejects one; `@Version` makes the loser fail with a 409.
6. **The entity doesn't exist** — `findById` returns an `Optional`; `orElseThrow` → 404 with a clean error body, never a 500 or a leaked `NullPointerException`.
7. **The model predicts badly** — it's a recommendation, not a guarantee; ranked output with confidence, and the fix is measurement, not hiding uncertainty.
8. **Invalid user input** — bean validation at the boundary → 400 with field messages before any business logic runs. Client-side validation is convenience, not security.
9. **The model service is down** — Spring catches the failure and returns 503; the rest of the app keeps working because auth, profiles and history don't depend on the model.
10. **The frontend and model disagree about field names** — the silent failure mode: the model returns a confident wrong answer with no exception anywhere. Validate the payload on both sides.

THE THREE 30-SECOND PROJECT SCRIPTS

REVERTPAY — escrow and payment reversal

"RevertPay is an escrow and payment-reversal platform. A buyer's money is held safely until the transaction completes: the buyer funds the escrow, the seller ships, the buyer confirms delivery, and only then are the funds released to the seller. I built it as a Spring Boot backend with PostgreSQL, secured with JWT and role-based authorization, plus a React frontend. The core design decision was to model the escrow as a validated state machine and to record money movements as ledger entries, so every state change is auditable."

The one-minute extension: the problem is trust between two strangers — if the buyer pays first the seller might not ship, and vice versa. Escrow solves it with a neutral hold. The escrow is a state machine, so an

illegal transition returns a conflict instead of corrupting the record. The money side is a ledger: a balance is derived as the sum of credits minus debits rather than stored, so it can never disagree with the history. Everything that changes money or state happens inside one transactional service method.

The hardest part: keeping the state change, the money movement and the audit record consistent — and safe under concurrency. One transaction, optimistic locking on a version column, and one central place where transitions are validated.

EDU2JOB — job-role recommendation (Infosys Springboard internship)

"During my Infosys Springboard internship I worked on EDU2JOB, a job-role recommendation system. A student enters their education, CGPA and skills, and a Random Forest model returns a ranked shortlist of suitable job roles. I built the Spring Boot backend with REST APIs for authentication, profiles, predictions and prediction history, integrated a React frontend, persisted everything in PostgreSQL, and worked with the Python scikit-learn model service that does the prediction."

The one-minute extension: the architecture is deliberately split. React calls Spring Boot and only Spring Boot. Spring handles JWT authentication, validates the input, then forwards the feature payload to a Python Flask service wrapping the trained Random Forest, which applies the same preprocessing used at training time and returns a probability per role. Spring stores the inputs and the raw response as prediction history and returns the top five roles. The browser never talks to Flask — one entry point means authentication and validation live in one place.

The honest limit: the model evaluation harness is still to do. I don't quote an accuracy number I can't verify.

ALGOVIZ — interactive algorithm visualization

"AlgoViz is an interactive algorithm visualization platform built with React, Vite, Tailwind and React Router. The key design decision is that algorithms record their execution as frames before playback, and the UI renders one recorded frame at a time. That makes pause, resume, speed control and step-by-step inspection trivial — they're all just moving a cursor — and it means you can seek backwards through the trace, which is impossible if you animate while the algorithm runs."

Why that matters: most algorithm animations show motion but not reasoning. Recording first separates execution from presentation, so each frame is a complete snapshot plus what's happening — which indices are compared, whether a swap occurred, and a message. Playback state lives in a reducer: cursor, playing, speed. Speed is just the delay between frames.

The trade-off I'll name before you ask: full snapshots make seeking easy but cost memory, so I capped the sorting input at 24 values instead of pretending it scales. Compact events or checkpoints are the fix if I lift the cap.

THE FINAL 15 MINUTES

Minute 0-3 — Java + OOP. JDK/JRE/JVM · `==` vs `equals()` and the hashCode contract · String immutability (security, pooling, safe map keys) · List/Set/Map and their complexities · interface vs abstract class · the four pillars, each with a project example.

Minute 3-6 — Spring + REST + JPA. Bean, IoC, DI, constructor injection · auto-configuration and the embedded server · CONTROLLER → SERVICE → REPOSITORY → DATABASE · the status codes · JPA spec, Hibernate implementation, Spring Data repositories · `@ManyToOne` owns the foreign key · LAZY vs EAGER · `@Transactional` is all-or-nothing.

Minute 6-8 — PostgreSQL + SQL. NUMERIC for money · primary keys auto-indexed, foreign keys not · indexes speed reads and slow writes · ACID · the transaction that groups state, ledger and audit · WHERE vs HAVING · INNER vs LEFT JOIN.

Minute 8-10 — JWT + Security. Authentication vs authorization · header.payload.signature · **signed, NOT encrypted** · the server verifies signature and expiry per request · the bearer header · BCrypt salted and slow · 401 vs 403 · roles plus ownership · CSRF disabled because the token is explicit · logout is client-side, with the honest limitation.

Minute 10-12 — React + JavaScript. Component · props vs state · never mutate state · `useEffect` dependencies · keys and why index keys break reordering · `useReducer` for related transitions · `map/filter/reduce` · `async/await` and checking `res.ok`. **Visualization: frames → cursor → render. Pause means stop scheduling the next cursor step.**

Minute 12-14 — Projects. Escrow is a neutral hold. A ledger with derived balances. One transaction per transition. React → Spring → Flask → model, and the browser never calls Flask.

Minute 14-15 — The openers. Self-introduction. Strongest project. Why hire me. **Then stop thinking and say them slowly.** Speed is not confidence.

THE NO-BLUFF RULE

When you don't know something, say this and stop talking:

"I haven't worked with that directly. What I know is [the concept one level above]. If I needed it, I'd [a concrete next step]."

That answer scores 3-4 out of 5. Bluffing scores 0 and poisons everything after it.

Three more things worth more than any definition:

1. **Name your limits before they find them.** A candidate who says "that's the weakest part of the project, and here's what I'd add" is trusted more than one who claims everything works.
2. **Volunteer the trade-off.** Every design decision has a cost. Saying it unprompted is the single biggest signal of real understanding.
3. **Slow down.** A correct answer delivered slowly beats a memorised answer delivered fast.

Built from my own resume in the hours before an interview. If it helps you, use it.